
Exploring Stratification Strategies for Reliable GNN Evaluation

Jonas Schmidt

Friedrich-Alexander-Universität Erlangen-Nürnberg
University Project Report

Abstract

Evaluation results for transductive node classification can depend strongly on the chosen train–validation–test split. Standard class stratification controls label prevalence but ignores that a graph neural network (GNN) receives information through graph topology. This project follows a staged search for graph-aware stratification properties. We first screened topology-only descriptors—degree, clustering coefficient, PageRank, and eigenvector centrality—and found dataset-specific gains but no consistent reduction in cross-fold standard deviation. We then examined three label–topology properties: neighborhood homogeneity, the full neighborhood-label distribution, and its unnormalized neighborhood count. Neighborhood homogeneity is the clearest positive result: it gives the lowest model-averaged dispersion on four of five high-homophily real-world graphs, with a Syn-Cora sweep providing controlled supporting evidence in the high-homophily regime. Motivated by label informativeness, neighborhood distribution and count were introduced to capture informative heterophily, but neither is consistently effective on low-homophily graphs; random or label stratification remains best on every low-homophily dataset in the model-averaged comparison. The project therefore yields a conditional conclusion rather than a universal splitter: graph-aware stratification helps when the selected neighborhood property reflects prediction difficulty, while weak task performance and uninformative neighborhood labels can make additional structure ineffective.

1 Introduction

Graph neural networks are commonly evaluated through transductive node classification: the full graph and node features are visible during training, while labels are disclosed only for designated training nodes. Although model design has advanced rapidly, the estimated test accuracy can remain fragile. [Shchur et al. \[2018\]](#) showed that changing data splits or training procedures can alter not only the reported score but also the ranking of architectures. Averaging over very many random splits reduces uncertainty, but it is expensive and does not address why some folds are systematically harder than others.

Ordinary stratified cross-validation preserves the marginal class distribution. For graphs, this is only part of the relevant sampling structure. Labels are statistically dependent along edges [\[Sen et al., 2008\]](#), and message passing makes a node’s prediction depend on a receptive field rather than on that node in isolation. Two folds with the same class counts may therefore contain different mixtures of locally easy, ambiguous, central, peripheral, homophilous, or heterophilous nodes.

This work asks whether balancing a model-agnostic description of node context can reduce the standard deviation of test accuracy across folds. Its contribution is not a search for a favorable split. Instead, it documents the full narrowing process: topology-only properties are screened first, the successful high-homophily signal is isolated, label-informativeness-inspired extensions are developed

for the low-homophily regime, and both positive and negative cases are evaluated across 11 graphs and six architectures.

2 From representative folds to candidate properties

2.1 Starting point: representative folds rather than a “best” split

The project began from the evaluation logic of [Jami et al. \[2025\]](#): a useful splitter should preserve relevant population structure in every fold and thereby stabilize the reported score. For graphs, one node is not an independent sample because its context is defined by connected nodes. The candidates were therefore organized by whether they use topology alone or combine topology with labels:

Table 1: Candidate properties considered during the project.

Group	Splitting properties
Baselines	Uniform random assignment; node-label stratification.
Topology-only	Degree, clustering coefficient, PageRank, and eigenvector centrality.
Label-topology	Neighborhood homogeneity, neighborhood-label distribution, and neighborhood-label count.

The topology-only properties describe structural position without consulting labels. The label-topology group instead describes which classes surround a node. The initial conjecture was that a useful graph-aware property would reduce dispersion beyond random and label stratification by distributing comparable node contexts across folds.

2.2 Why label-topology interactions became central

Neighborhood homogeneity measures the same-label fraction around an individual node, while label informativeness (LI) measures how much a neighbor’s label reduces uncertainty about the target label [[Platonov et al., 2023](#)]. LI is important because two graphs can have similar homophily but different class-transition patterns. The label-position results of [Han et al. \[2023\]](#) likewise show that graph evaluation difficulty can depend on where labels occur relative to a node. These observations motivated a staged empirical question: do topology-only properties suffice; if not, does neighborhood-homogeneity stratification work in highly homophilous graphs, where most edges connect nodes of the same class; and can richer class-transition representations help when homophily is low?

3 Screening topology-only properties

Figure 1 condenses the first empirical screen. The result is mixed rather than uniformly negative. For example, eigenvector centrality substantially reduces model-averaged dispersion on Chameleon and Cora, whereas it is neutral or detrimental on CiteSeer and Actor. Degree helps on Chameleon and Texas but slightly worsens Cora and Actor. Clustering coefficient is especially inconsistent. Thus the topology-only stage did not identify a property whose benefit transferred reliably across datasets and architectures. This motivated concentrating on properties that couple topology to the label configurations actually propagated by a GNN.

3.1 Neighborhood homogeneity in the high-homophily regime

The next screen focused on neighborhood homogeneity. Table 2 shows the model-averaged result for the five high-homophily real-world datasets. The property obtains the lowest dispersion on Cora, CiteSeer, PubMed, and Photo; Computers is the exception, where label stratification is stronger at this stage.

Syn-Cora supplies a controlled check of this finding. The benchmark varies homophily while retaining Cora-derived labels and features [[Zhu et al., 2020](#)]. In the high-homophily part of our sweep, neighborhood-homogeneity stratification again lowered cross-fold dispersion relative to the random and label baselines. This supports the interpretation that the real-world result is tied to the

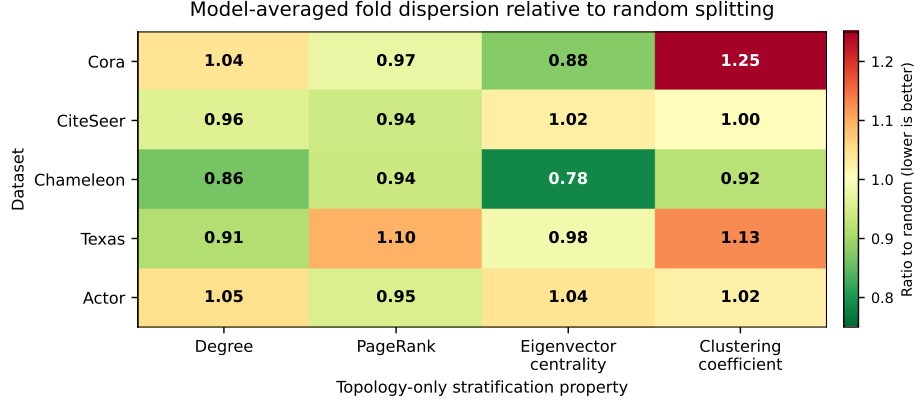


Figure 1: Topology-only screening. Each cell is the model-averaged fold standard deviation under the indicated stratifier divided by the random-split value. Values below one are improvements. Individual properties help on individual datasets, but no topology-only property gives a consistent reduction across the five datasets for which the complete screen was run. Full per-model tables are reported in Appendix C.

Table 2: Model-averaged fold standard deviation on high-homophily graphs (percentage points; lower is better).

Dataset	Random	Label	NEIGH.-HOM.
Cora	1.405	1.588	1.130
CiteSeer	1.680	1.508	1.315
PubMed	0.480	0.780	0.455
Computers	1.005	0.590	0.702
Photo	0.832	0.387	0.325

homophily regime rather than to one particular citation graph. It also sets up the remaining challenge: a same-label fraction becomes less expressive when same-label edges are rare.

4 LI-inspired label-topology extensions

Let $G = (V, E)$ be an undirected graph, let $y_v \in \{1, \dots, C\}$ denote a node label, and let $e_{y_v} \in \{0, 1\}^C$ be its one-hot encoding. For node v , the unnormalized neighborhood-label count and normalized distribution are

$$c_v = \sum_{u \in \mathcal{N}(v)} e_{y_u}, \quad q_v = \frac{c_v}{\max(|\mathcal{N}(v)|, 1)}. \quad (1)$$

Neighborhood homogeneity is the single component q_{v, y_v} . In contrast, NEIGH.-DIST. retains all C class proportions, distinguishing nodes with the same homogeneity but different surrounding classes. NEIGH.-COUNT uses c_v directly, preserving the density/sparsity information removed by normalization. This distinction was motivated by LI: in a heterophilous graph, the identity of a different-class neighbor can still be informative even when the same-label fraction is near zero.

To reflect a multi-hop receptive field, the normalized representation can be extended with row-normalized adjacency $P = D^{-1}A$ and label matrix Y :

$$H^{(0)} = Y, \quad H^{(k)} = PH^{(k-1)}, \quad \tilde{q}_v = \sum_{k=1}^K \alpha^{k-1} H_{v, :}^{(k)}, \quad q_v = \frac{\tilde{q}_v}{\sum_c \tilde{q}_{v, c}}. \quad (2)$$

The sum begins at $k = 1$, excluding the target node’s own label. The resulting continuous vectors are clustered with k -means, and the number of clusters is selected with the gap statistic [Tibshirani et al., 2001]. Cluster identifiers become strata for ordinary fold allocation.

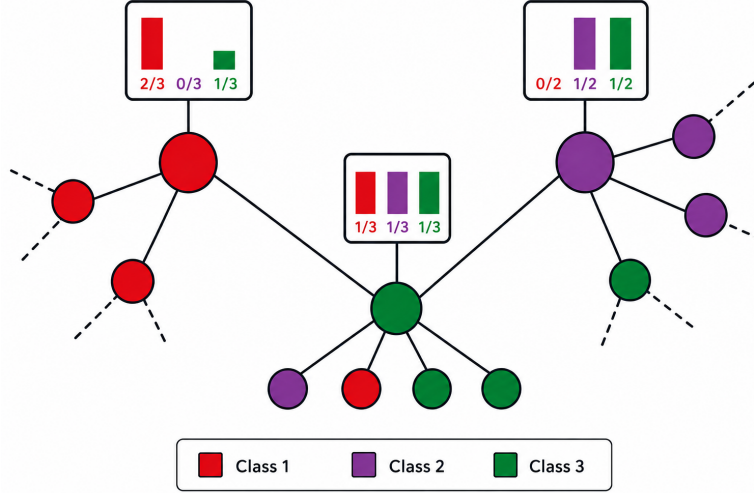


Figure 2: Nodes with different labels can occupy different local class mixtures. Neighborhood homogeneity retains only the same-label share; neighborhood distribution retains the complete class composition, and neighborhood count additionally retains neighborhood size.

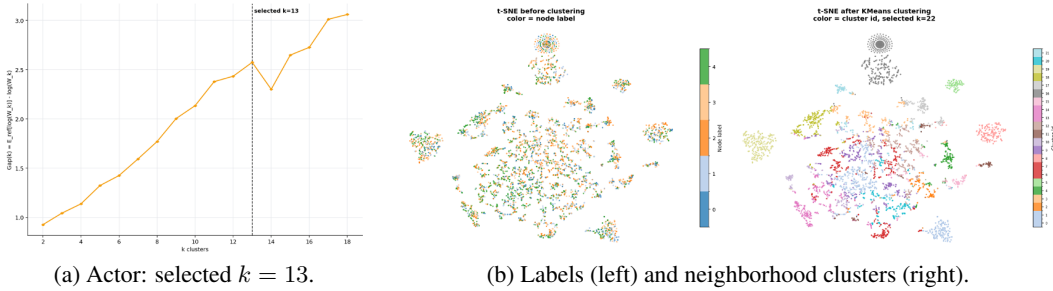


Figure 3: Cluster diagnostics. The gap statistic supplies a reproducible cluster-count rule. The t-SNE projection [van der Maaten and Hinton, 2008] is explanatory only and is not used by the splitter.

5 Experimental design

5.1 Protocol and models

All experiments use five folds throughout. The split construction is specified here and implemented by the rotating-mask procedure described in Section 5.1. For fold i , fold i is test data, fold $(i + 1) \bmod 5$ is validation data, and the remaining three folds are training data. Thus every rotation uses three folds for training, one for validation, and one for testing. All splitting strategies use the same fold proportions and rotating masks.

To obtain robust estimates, we repeat the complete splitting and evaluation procedure for $n = 10$ independent seeds. For every seed, we compute the standard deviation of test accuracy across the five folds and subsequently average this value across the 10 independent seedings. Reported dispersion values are therefore seed-averaged five-fold standard deviations in percentage points; lower is better. Mean accuracy remains a complementary sanity check, while the scientific claim concerns dispersion.

Six models cover complementary propagation assumptions: GCN [Kipf and Welling, 2017], GAT [Veličković et al., 2018], GraphSAGE [Hamilton et al., 2017], GPRGNN [Chien et al., 2021], H2GCN [Zhu et al., 2020], and an MLP that ignores all graph edges. The MLP is a negative control: changes in its dispersion reveal correlations between a graph-derived stratum, node features, and intrinsic node difficulty rather than a pure message-passing effect.

Table 3: Datasets span homophilic citation/co-purchase graphs and heterophilic web, actor, and historical networks. h is the reported homophily score and LI is label informativeness.

Dataset	Nodes	Edges	Classes	h	LI
Cora	2,708	5,278	7	.81	.612
CiteSeer	3,327	4,552	6	.74	.432
PubMed	19,717	44,324	3	.80	.391
Computers	13,752	245,861	10	.78	.598
Photo	7,650	119,081	8	.83	.697
Chameleon	890	8,854	5	.23	.025
Squirrel	2,223	46,998	5	.22	.001
Wisconsin	251	450	5	.21	.152
Texas	183	279	5	.11	.234
Actor	7,600	26,659	5	.22	.000
Roman Empire	22,662	32,927	18	.05	.112

5.2 Training duration and the need for early stopping

Before fixing the protocol, we considered training every model for exactly 1,000 epochs without a validation set. This would simplify the split and make more labeled nodes available for training, but it could also retain over-trained or still-transient models. We therefore measured how much performance would be lost by relying on a fixed horizon alone. Figure 4 shows that Cora is largely stable after early training, whereas Chameleon exhibits persistent architecture differences and a substantial early transient for GPRGNN. The experiment therefore led us to retain a validation fold and use early stopping with a generous maximum epoch budget; a fixed 1,000-epoch schedule without validation was not considered sufficiently reliable.

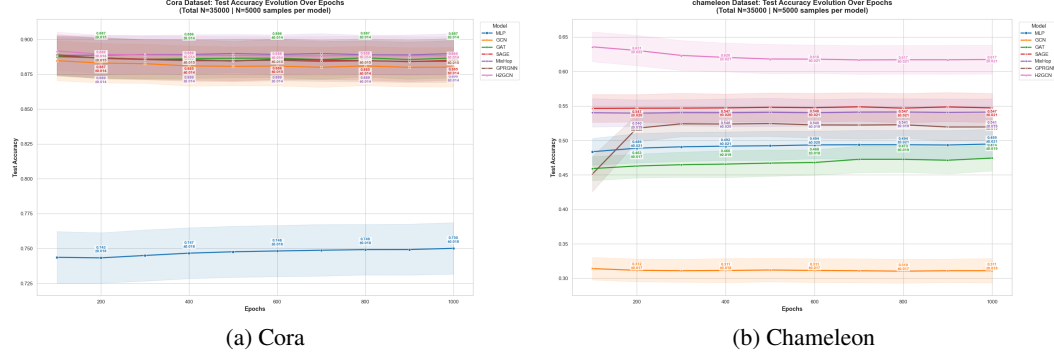


Figure 4: Test-accuracy trajectories over 1,000 epochs. Shaded regions show empirical dispersion. Cora is insensitive to late training, whereas Chameleon exposes large and persistent model differences.

The earlier dense-versus-sparse labeling study is reported in Appendix A. It showed that reducing the training share to 10% simultaneously lowers performance and increases aggregate dispersion, so fold proportions were held fixed in all subsequent property comparisons.

5.3 Label balance does not imply topology balance

Before directly optimizing graph properties, we compared train and test distributions under ordinary node-label stratification. Figure 5 shows representative folds for Cora and Chameleon. The main modes often overlap, but tail mass and secondary modes can differ, especially for degree and centrality. This confirms that preserving class prevalence does not automatically preserve structural position. It also explains why the topology-only candidates were reasonable to test even though they ultimately failed to transfer consistently.

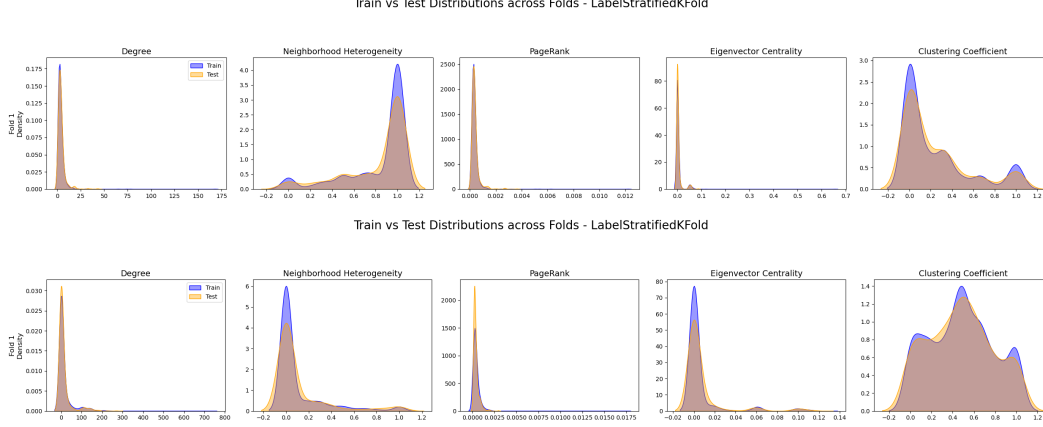


Figure 5: First-fold train (blue) and test (orange) structural densities under node-label stratification for Cora (top) and Chameleon (bottom). Label balance does not guarantee identical topology-related distributions.

6 Main results

Table 4 averages the fold-level test-accuracy standard deviation over the six models for each dataset. This view prevents any one architecture from defining the conclusion. The pattern is sharply regime-dependent: a label-topology property wins every high-homophily dataset, while none wins a low-homophily dataset.

Table 4: Five-fold test-accuracy standard deviation, first computed within each of 10 seeds, averaged across seeds, and then averaged across the six models (percentage points; lower is better). Green marks the best strategy per dataset. A dash denotes an unavailable run.

Dataset	Random	Label	NEIGH.-HOM.	NEIGH.-DIST.	NEIGH.-COUNT
Cora	1.405	1.588	1.130	1.168	1.420
CiteSeer	1.680	1.508	1.315	1.447	1.758
PubMed	0.480	0.780	0.455	0.547	—
Computers	1.005	0.590	0.702	0.497	0.560
Photo	0.832	0.387	0.325	0.677	0.427
Chameleon	2.548	2.138	2.762	3.442	3.363
Squirrel	1.802	1.883	1.932	1.890	2.593
Wisconsin	5.315	5.525	6.612	6.527	6.262
Texas	7.465	6.005	7.965	7.032	6.440
Actor	1.052	0.948	1.208	1.297	1.068
Roman Empire	0.733	0.558	0.677	0.877	0.633

6.1 High homophily: a repeatable neighborhood signal

Neighborhood homogeneity is best on four of the five high-homophily real-world graphs. Computers is the useful exception: NEIGH.-DIST. is best, suggesting that the identity of surrounding classes contains information that the same-label fraction discards. Together with the Syn-Cora check, these results support neighborhood homogeneity as a suitable stratification property in the high-homophily regime, but not as a universal rule.

6.2 Low homophily: richer LI-inspired properties do not transfer consistently

The full neighborhood distribution and count were introduced precisely because same-label homogeneity is compressed near zero in heterophilous graphs. The conjecture was that different-class transition patterns, which LI can capture globally, would identify locally comparable test nodes. The

results do not support that conjecture consistently. Label stratification is best on Chameleon, Texas, Actor, and Roman Empire; random splitting is best on Squirrel and Wisconsin. Neither NEIGH.-DIST. nor NEIGH.-COUNT is best on any low-homophily dataset after averaging across models.

One plausible explanation is that a splitter can balance only a property that the evaluated models can exploit. Several low-homophily datasets show low and strongly architecture-dependent predictive performance. In that setting, neighborhood-label geometry may be real but weakly related to the remaining classification error. Hard clustering also creates small or unstable strata, particularly on small graphs such as Texas and Wisconsin. The negative result is therefore informative: more detailed neighborhood statistics are not automatically more useful evaluation strata.

7 Cross-model checks

The dataset averages do not hide a universally successful architecture-specific result. Across the 66 dataset–model cells, label stratification and neighborhood homogeneity account for most row minima. Table 5 credits every method involved in a tie, which is why the counts sum to more than 66. The LI-inspired distribution and count win substantially fewer cells than either label stratification or neighborhood homogeneity.

Table 5: Number of lowest-dispersion dataset–model cells. Tied minima are credited to each tied method.

Method	Random	Label	NEIGH.-HOM.	NEIGH.-DIST.	NEIGH.-COUNT
Cell minima	9	23	21	10	7

Computers is the main positive exception for NEIGH.-DIST.: GCN, GAT, and GraphSAGE all achieve their lowest dispersion with the full neighborhood distribution. The effect does not extend uniformly to GPRGNN, H2GCN, or the MLP, and it does not recur across the low-homophily group. This reinforces the central interpretation that the usefulness of a stratum depends jointly on dataset regime and model behavior.

The MLP remains a critical negative control. Graph-derived strata change MLP dispersion in several cells even though the MLP never consumes edges. Neighborhood properties can correlate with node features, class difficulty, or sampling density. A graph-aware improvement should therefore be interpreted alongside the MLP response rather than automatically attributed to message passing.

8 Interpretation and discussion

The project trajectory. The search followed a coherent narrowing. Topology-only properties were plausible descriptors of structural position, but their effects were dataset-specific. Neighborhood homogeneity then provided a stable signal in high-homophily graphs. NEIGH.-DIST. and NEIGH.-COUNT extended this idea using the class-transition perspective of LI, but did not solve the low-homophily regime. The final contribution is therefore a map of where these properties work, not a claim that one splitter dominates everywhere.

Why the topology screen failed is not a collateral-balance artifact. We explicitly tested whether optimizing one property merely destroyed the balance of the others. Appendix B shows that the targeted property’s fold discrepancy falls strongly, while non-targeted discrepancies remain close to their random-fold baselines. Thus the inconsistent test-accuracy effects are better explained by the relevance of the chosen property than by systematic damage to the remaining properties.

Limitations. The experimental sweep is necessarily uneven: some large datasets were run with fewer split seeds, and several topology-only combinations were not completed. The screening and final label–topology stages also represent successive configurations, so their neighborhood-homogeneity cells should not be compared as if they were one pooled run. Differences between splitters do not yet have paired confidence intervals. Finally, the cluster count, hop depth, geometric decay, and count normalization are method choices that must be selected without test accuracy.

Next step. Future work should not assume that one graph-aware splitter works for every dataset. A simple next step is to preserve the class-label distribution first and, when the graph has a useful label–topology relationship, additionally balance neighborhood distributions within each class. Instead of assigning nodes to fixed neighborhood clusters, a future method could directly minimize the difference between the neighborhood distributions of the folds. Random and label stratification should remain required baselines.

9 Practical selection rule

The experiments suggest treating the choice of splitter as a small model-selection problem rather than fixing a single graph-aware rule for every benchmark. The decision must be made without consulting final test accuracy: use graph-level diagnostics and a pilot validation analysis, archive the choice, and then evaluate every model on the same folds.

Table 6: Observed conditions and their implications for selecting a splitting strategy.

Diagnostic	Interpretation
High homophily and stable one-hop bins	Compare neighborhood homogeneity with label stratification; the project evidence favors neighborhood homogeneity in this regime.
Low homophily or near-zero LI	Retain random and label stratification as strong defaults; richer neighborhood classes are not automatically informative.
Large MLP response	The split also changes feature or intrinsic difficulty; avoid attributing the entire effect to message passing.

10 Conclusion

This project investigated whether graph-aware folds can stabilize transductive GNN evaluation. Purely topological properties did not yield a consistent reduction in cross-fold standard deviation. Neighborhood homogeneity did: it is the strongest method on four of five high-homophily real-world datasets, and Syn-Cora supports the same conclusion under controlled homophily. The LI-inspired neighborhood distribution and count were reasonable attempts to extend this success to heterophily, yet neither consistently improves the low-homophily datasets. The lean final conclusion is therefore regime-specific: balance neighborhood homogeneity when the graph is strongly homophilous and the signal is empirically verified; otherwise, treat random and label stratification as strong defaults and do not assume that richer graph structure produces a more reliable evaluation split.

References

- Eli Chien, Jianhao Peng, Pan Li, and Olgica Milenkovic. Adaptive universal generalized pagerank graph neural network. In *International Conference on Learning Representations*, 2021.
- William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- Haoyu Han, Xiaorui Liu, Feng Shi, MohamadAli Torkamani, Charu C. Aggarwal, and Jiliang Tang. Towards label position bias in graph neural networks. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Naga Venkata Sai Jitin Jami, Thomas Altstidl, Jonas Leo Mueller, Jindong Li, Dario Zanca, Bjorn Eskofier, and Heike Leutheuser. Stratify or die: Rethinking data splits in image segmentation. In *Advances in Neural Information Processing Systems*, volume 38, 2025.
- Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017.
- Oleg Platonov, Denis Kuznedelev, Artem Babenko, and Liudmila Prokhorenkova. Characterizing graph datasets for node classification: Homophily–heterophily dichotomy and beyond. In *Advances in Neural Information Processing Systems*, volume 36, pages 523–548, 2023.
- Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Gallagher, and Tina Eliassi-Rad. Collective classification in network data. *AI Magazine*, 29(3):93–106, 2008.
- Oleksandr Shchur, Maximilian Mumme, Aleksandar Bojchevski, and Stephan Günnemann. Pitfalls of graph neural network evaluation. *Relational Representation Learning Workshop, NeurIPS*, 2018.
- Robert Tibshirani, Guenther Walther, and Trevor Hastie. Estimating the number of clusters in a data set via the gap statistic. *Journal of the Royal Statistical Society: Series B*, 63(2):411–423, 2001.
- Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9:2579–2605, 2008.
- Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations*, 2018.
- Jiong Zhu, Yujun Yan, Lingxiao Zhao, Mark Heimann, Leman Akoglu, and Danai Koutra. Beyond homophily in graph neural networks: Current limitations and effective designs. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

A Dense and sparse labeling regimes

Before screening stratification properties, we examined whether the amount of labeled training data changes both predictive performance and fold dispersion. The pilot compared train/validation/test ratios of 60/20/20, 50/30/20, and 10/20/30 on Cora and CiteSeer. The two dense-label settings are similar. In contrast, the sparse 10% training regime lowers model-wise and aggregate test accuracy and produces a visibly wider aggregate distribution. This matters methodologically: a change attributed to stratification can otherwise be confounded with the difficulty created by label scarcity. We therefore fixed the fold proportions before comparing node properties.

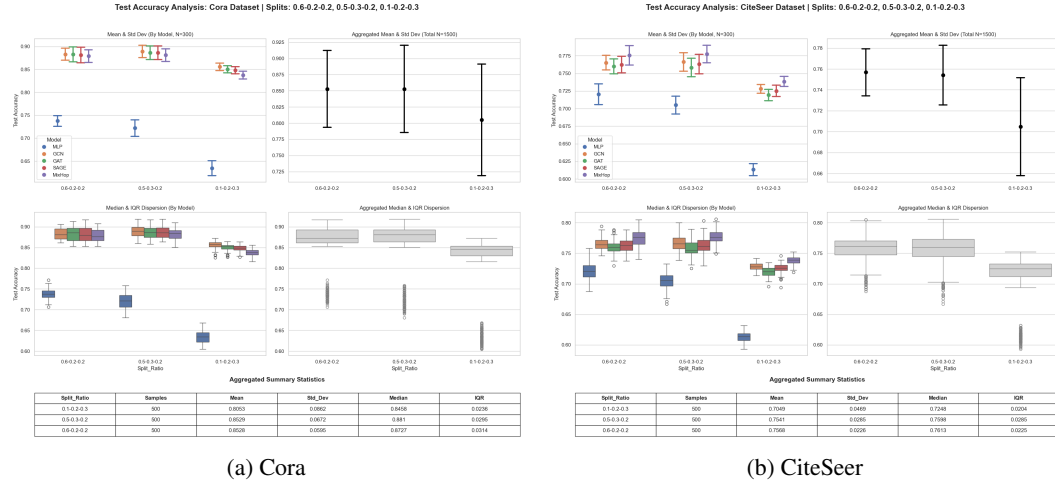


Figure 6: Dense-versus-sparse labeling pilot. Each panel reports model-wise means and standard deviations, aggregate mean and standard deviation, model-wise median/IQR distributions, and the aggregate distribution. The 10/20/30 split is consistently more difficult than the two dense-label settings.

B Cross-property preservation

The topology-only screen raised a possible alternative explanation: perhaps one property appears better only because optimizing it severely worsens the remaining structural properties. We tested this directly on Cora. For each row in Figure 7, one property was optimized and every column was monitored. Values report the random-fold discrepancy divided by the achieved discrepancy; a diagonal value above one therefore indicates successful improvement in the targeted property, while an off-diagonal value near one indicates that a non-targeted property stayed approximately unchanged.

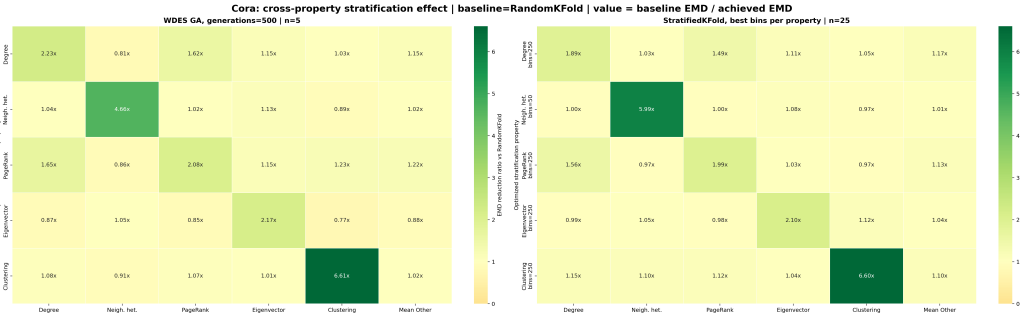


Figure 7: Cross-property influence on Cora for the genetic/Wasserstein prototype (left) and stratified-fold construction (right). Large diagonal values show that each target can be balanced. Off-diagonal values remain comparatively close to one, so improving one property generally does not destroy the others. “Mean Other” summarizes the non-targeted columns.

This result rules out a simple collateral-damage explanation. The split optimizer can materially reduce the discrepancy of its target while leaving the other monitored properties relatively stable. Consequently, the downstream differences in accuracy dispersion are more plausibly caused by whether the target property is related to node-level prediction difficulty.

C Detailed topology-only screening tables

The following tables reproduce the complete topology-screening results supplied during the project. Each cell is the standard deviation across fold test accuracies in percentage points; lower is better. Dashes denote combinations that were not run. These tables belong to the earlier screening stage. The later label-topology benchmark in Table 4 used the finalized configuration, so neighborhood-homogeneity values should be interpreted within their respective stage rather than pooled across stages.

Table 7: Cora topology screening. Zhu $h = 0.810$; 2,708 nodes; 5,278 edges; average degree 3.90; LI 0.612.

Model	Random	Label	Degree	Neigh. Hom.	PageRank	Eigenvector	Clustering
GCN	1.18	1.45	1.29	0.78	1.46	1.10	1.86
GAT	1.26	1.43	1.38	0.86	1.61	1.29	1.67
SAGE	1.42	1.58	1.07	0.97	1.05	1.31	1.78
GPRGNN	1.28	1.46	1.22	1.04	1.53	1.18	1.51
H2GCN	1.36	1.59	1.34	0.84	1.01	1.08	1.72
MLP	1.93	2.02	2.48	2.29	1.54	1.42	1.97

Table 8: CiteSeer topology screening. Zhu $h = 0.740$; 3,327 nodes; 4,552 edges; average degree 2.74; LI 0.432.

Model	Random	Label	Degree	Neigh. Hom.	PageRank	Eigenvector	Clustering
GCN	1.80	1.56	1.64	1.31	1.48	1.64	1.63
GAT	1.71	1.48	1.69	1.24	1.58	1.69	1.63
SAGE	1.74	1.55	1.66	1.34	1.57	1.84	1.60
GPRGNN	1.68	1.55	1.56	1.28	1.49	1.81	1.80
H2GCN	1.74	1.57	1.50	1.17	1.64	1.66	1.57
MLP	1.41	1.34	1.61	1.55	1.69	1.69	1.87

Table 9: PubMed topology screening. Zhu $h = 0.800$; 19,717 nodes; 44,324 edges; average degree 4.50; LI 0.391.

Model	Random	Label	Degree	Neigh. Hom.	PageRank	Eigenvector	Clustering
GCN	0.41	0.76	0.83	0.40	0.52	0.88	1.02
GAT	0.56	0.79	0.61	0.64	0.55	0.87	0.72
SAGE	0.54	0.90	0.73	0.49	0.65	0.92	0.72
GPRGNN	0.38	0.93	0.43	1.10	0.98	0.79	0.85
H2GCN	0.45	0.63	0.87	0.39	1.03	0.65	0.89
MLP	0.54	0.67	0.56	0.72	1.02	0.53	0.92

Table 10: Computers topology screening. Edge homophily 0.780; 13,752 nodes; 245,861 edges; average degree 35.76; LI 0.598.

Model	Random	Label	Degree	Neigh. Hom.	PageRank	Eigenvector	Clustering
GCN	0.39	0.37	1.04	0.42	0.82	0.68	1.22
GAT	0.30	0.37	0.45	0.31	0.56	0.85	0.78
SAGE	3.54	1.15	1.23	0.91	1.32	2.41	0.95
GPRGNN	0.26	0.31	0.45	0.32	0.45	0.37	0.59
H2GCN	1.08	0.49	0.67	0.42	1.23	0.45	0.85
MLP	0.46	0.85	1.19	0.51	1.03	0.45	0.54

Table 11: Photo topology screening. Edge homophily 0.830; 7,650 nodes; 119,081 edges; average degree 31.13; LI 0.697.

Model	Random	Label	Degree	Neigh. Hom.	PageRank	Eigenvector	Clustering
GCN	1.06	0.18	—	—	—	—	—
GAT	0.78	0.47	—	—	—	—	—
SAGE	0.82	0.33	—	—	—	—	—
GPRGNN	0.77	0.29	—	—	—	—	—
H2GCN	0.92	0.29	—	—	—	—	—
MLP	0.64	0.76	—	—	—	—	—

Table 12: Chameleon topology screening. Zhu $h = 0.230$; 890 nodes; 8,854 edges; average degree 19.90; LI 0.025.

Model	Random	Label	Degree	Neigh. Hom.	PageRank	Eigenvector	Clustering
GCN	2.78	2.15	1.93	2.30	2.17	1.60	2.16
GAT	2.23	2.07	2.06	2.14	2.34	2.22	2.38
SAGE	2.30	1.81	2.02	2.14	2.32	1.82	2.63
GPRGNN	2.85	2.43	2.48	2.77	1.99	2.22	2.25
H2GCN	2.59	2.42	2.64	2.21	2.78	1.81	2.38
MLP	2.54	1.95	2.07	2.40	2.71	2.32	2.26

Table 13: Squirrel topology screening. Zhu $h = 0.220$; 2,223 nodes; 46,998 edges; average degree 42.28; LI 0.001.

Model	Random	Label	Degree	Neigh. Hom.	PageRank	Eigenvector	Clustering
GCN	2.04	1.99	3.01	1.76	2.05	1.89	2.21
GAT	1.51	1.33	1.76	1.85	1.86	1.43	1.96
SAGE	1.43	1.84	1.76	1.45	2.21	2.07	1.83
GPRGNN	1.82	2.01	1.98	1.94	2.06	1.98	1.65
H2GCN	2.21	1.55	2.01	1.78	3.10	1.83	2.23
MLP	1.80	1.98	1.87	2.02	2.45	1.99	2.56

Table 14: Wisconsin topology screening. Zhu $h = 0.210$; 251 nodes; 450 edges; average degree 3.59; LI 0.152.

Model	Random	Label	Degree	Neigh. Hom.	PageRank	Eigenvector	Clustering
GCN	6.51	7.01	6.71	6.70	7.11	8.23	6.74
GAT	5.84	5.61	5.89	6.49	8.43	4.21	5.01
SAGE	4.87	5.24	7.01	4.98	6.38	6.35	5.44
GPRGNN	6.41	6.60	4.21	4.73	5.84	6.06	5.32
H2GCN	3.71	4.80	5.50	4.32	5.21	7.95	3.51
MLP	4.55	3.89	5.21	3.67	3.90	4.29	6.02

Table 15: Texas topology screening. Zhu $h = 0.110$; 183 nodes; 279 edges; average degree 3.05; LI 0.234.

Model	Random	Label	Degree	Neigh. Hom.	PageRank	Eigenvector	Clustering
GCN	9.32	5.56	5.76	8.31	7.40	7.26	7.51
GAT	8.94	4.19	6.38	7.34	7.72	7.42	8.82
SAGE	8.63	7.00	6.70	7.57	8.87	8.33	9.69
GPRGNN	6.65	9.28	9.44	9.52	11.57	7.29	11.90
H2GCN	5.86	5.33	6.61	7.91	7.30	7.72	6.98
MLP	5.39	4.67	6.02	6.15	6.19	6.07	5.57

Table 16: Actor topology screening. Zhu $h = 0.220$; 7,600 nodes; 26,659 edges; average degree 7.02; LI 0.000.

Model	Random	Label	Degree	Neigh. Hom.	PageRank	Eigenvector	Clustering
GCN	0.96	0.67	0.96	1.23	0.89	1.15	1.26
GAT	1.13	0.94	1.16	1.23	0.85	1.19	1.17
SAGE	1.04	1.12	0.97	1.08	1.00	0.76	1.09
GPRGNN	1.14	0.97	1.35	1.27	1.06	1.13	0.95
H2GCN	1.01	1.05	1.00	1.16	1.22	1.02	1.03
MLP	1.03	0.94	1.17	1.15	0.99	1.33	0.95

Table 17: Roman Empire topology screening. Edge homophily 0.050; 22,662 nodes; 32,927 edges; average degree 2.91; LI 0.112.

Model	Random	Label	Degree	Neigh. Hom.	PageRank	Eigenvector	Clustering
GCN	0.69	0.57	0.72	0.98	0.62	0.59	1.23
GAT	0.83	0.90	1.21	0.65	0.77	0.72	0.98
SAGE	0.58	0.66	0.61	1.03	1.01	0.54	0.94
GPRGNN	0.72	0.48	0.76	0.82	0.55	0.96	0.56
H2GCN	0.86	0.83	1.02	1.31	0.71	0.84	0.69
MLP	0.72	0.81	0.75	0.87	0.71	1.12	0.93

D Detailed label–topology results

The following tables report the finalized label–topology comparison for every dataset and model. For each cell, the standard deviation across the five rotating folds is computed separately for each of 10 independent seeds and then averaged across seeds. Values are percentage points; lower is better. A dash denotes an unavailable run.

Table 18: Cora: aggregated test-accuracy dispersion. Each cell reports the standard deviation across fold test accuracies. Zhu h=0.810; nodes=2708 | edges=5278 | avg_deg=3.90; LI=0.612.

Model	Random	Label	Neigh. Hom.	Neigh. Distribution	Neigh. Count
GCN	1.18	1.45	1.14	1.19	1.49
GAT	1.26	1.43	1.21	1.63	1.36
SAGE	1.42	1.58	1.32	1.15	1.41
GPRGNN	1.28	1.46	1.10	0.99	1.44
H2GCN	1.36	1.59	1.08	1.10	1.39
MLP	1.93	2.02	1.75	1.48	1.43

Table 19: CiteSeer: aggregated test-accuracy dispersion. Each cell reports the standard deviation across fold test accuracies. Zhu h=0.740; nodes=3327 | edges=4552 | avg_deg=2.74; LI=0.432.

Model	Random	Label	Neigh. Hom.	Neigh. Distribution	Neigh. Count
GCN	1.80	1.56	1.54	1.36	2.00
GAT	1.71	1.48	1.78	1.21	1.80
SAGE	1.74	1.55	1.61	1.57	1.67
GPRGNN	1.68	1.55	1.77	1.57	1.74
H2GCN	1.74	1.57	1.82	1.39	1.67
MLP	1.41	1.34	1.41	1.58	1.67

Table 20: PubMed: aggregated test-accuracy dispersion. Each cell reports the standard deviation across fold test accuracies. Zhu h=0.800; nodes=19717 | edges=44324 | avg_deg=4.50; LI=0.391.

Model	Random	Label	Neigh. Hom.	Neigh. Distribution	Neigh. Count
GCN	0.41	0.76	0.40	0.50	0.43
GAT	0.56	0.79	0.29	0.59	0.61
SAGE	0.54	0.90	0.39	0.56	0.75
GPRGNN	0.38	0.93	0.57	0.64	0.44
H2GCN	0.45	0.63	0.41	0.49	0.71
MLP	0.54	0.67	0.67	0.50	0.50

Table 21: Computers: aggregated test-accuracy dispersion. Each cell reports the standard deviation across fold test accuracies. $h_edge=0.780$; nodes=13752 | edges=245861 | $avg_deg=35.76$; $LI=0.598$.

Model	Random	Label	Neigh. Hom.	Neigh. Distribution	Neigh. Count
GCN	0.39	0.37	0.57	0.30	0.61
GAT	0.30	0.37	0.48	0.29	0.51
SAGE	3.54	1.15	1.57	0.58	0.61
GPRGNN	0.26	0.31	0.34	0.37	0.76
H2GCN	1.08	0.49	0.64	0.82	0.42
MLP	0.46	0.85	0.61	0.62	0.45

Table 22: Photo: aggregated test-accuracy dispersion. Each cell reports the standard deviation across fold test accuracies. $h_edge=0.830$; nodes=7650 | edges=119081 | $avg_deg=31.13$; $LI=0.697$.

Model	Random	Label	Neigh. Hom.	Neigh. Distribution	Neigh. Count
GCN	1.06	0.18	0.27	0.60	0.42
GAT	0.78	0.47	0.23	0.65	0.26
SAGE	0.82	0.33	0.40	0.64	0.33
GPRGNN	0.77	0.29	0.04	0.62	0.26
H2GCN	0.92	0.29	0.46	0.96	0.64
MLP	0.64	0.76	0.55	0.59	0.65

Table 23: chameleon: aggregated test-accuracy dispersion. Each cell reports the standard deviation across fold test accuracies. $Zhu\ h=0.230$; nodes=890 | edges=8854 | $avg_deg=19.90$; $LI=0.025$.

Model	Random	Label	Neigh. Hom.	Neigh. Distribution	Neigh. Count
GCN	2.78	2.15	2.89	3.05	3.30
GAT	2.23	2.07	3.47	3.32	2.94
SAGE	2.30	1.81	2.90	3.55	3.31
GPRGNN	2.85	2.43	2.74	3.67	3.79
H2GCN	2.59	2.42	3.40	3.28	3.71
MLP	2.54	1.95	3.73	3.78	3.13

Table 24: squirrel: aggregated test-accuracy dispersion. Each cell reports the standard deviation across fold test accuracies. $Zhu\ h=0.220$; nodes=2223 | edges=46998 | $avg_deg=42.28$; $LI=0.001$.

Model	Random	Label	Neigh. Hom.	Neigh. Distribution	Neigh. Count
GCN	2.04	1.99	1.99	2.43	2.75
GAT	1.51	1.33	1.65	1.75	2.24
SAGE	1.43	1.84	2.02	1.80	2.74
GPRGNN	1.82	2.01	1.54	2.18	2.43
H2GCN	2.21	2.15	2.17	1.44	2.57
MLP	1.80	1.98	2.22	1.74	2.83

Table 25: wisconsin: aggregated test-accuracy dispersion. Each cell reports the standard deviation across fold test accuracies. $Zhu\ h=0.210$; nodes=251 | edges=450 | $avg_deg=3.59$; $LI=0.152$.

Model	Random	Label	Neigh. Hom.	Neigh. Distribution	Neigh. Count
GCN	6.51	7.01	7.17	7.99	8.52
GAT	5.84	5.61	7.84	7.67	8.46
SAGE	4.87	5.24	5.88	6.63	6.02
GPRGNN	6.41	6.60	7.56	7.22	6.74
H2GCN	3.71	4.80	5.64	4.68	4.30
MLP	4.55	3.89	5.58	4.97	3.53

Table 26: Texas: aggregated test-accuracy dispersion. Each cell reports the standard deviation across fold test accuracies. Zhu h=0.110; nodes=183 | edges=279 | avg_deg=3.05; LI=0.234.

Model	Random	Label	Neigh. Hom.	Neigh. Distribution	Neigh. Count
GCN	9.32	5.56	8.46	7.44	6.31
GAT	8.94	4.19	9.00	9.21	6.42
SAGE	8.63	7.00	7.77	6.90	8.04
GPRGNN	6.65	9.28	9.72	6.51	7.58
H2GCN	5.86	5.33	7.41	6.66	6.02
MLP	5.39	4.67	6.40	5.47	4.27

Table 27: Actor: aggregated test-accuracy dispersion. Each cell reports the standard deviation across fold test accuracies. Zhu h=0.220; nodes=7600 | edges=26659 | avg_deg=7.02; LI=0.000.

Model	Random	Label	Neigh. Hom.	Neigh. Distribution	Neigh. Count
GCN	0.96	0.67	1.26	1.33	1.18
GAT	1.13	0.94	1.13	1.48	1.02
SAGE	1.04	1.12	1.01	1.37	1.05
GPRGNN	1.14	0.97	1.48	1.28	1.00
H2GCN	1.01	1.05	1.24	1.14	1.07
MLP	1.03	0.94	1.29	1.18	1.09

Table 28: roman_empire: aggregated test-accuracy dispersion. Each cell reports the standard deviation across fold test accuracies. h_edge=0.050; nodes=22662 | edges=32927 | avg_deg=2.91; LI=0.112.

Model	Random	Label	Neigh. Hom.	Neigh. Distribution	Neigh. Count
GCN	0.69	0.57	0.90	0.84	0.60
GAT	0.83	0.50	0.90	1.09	0.63
SAGE	0.58	0.46	0.34	0.72	0.78
GPRGNN	0.72	0.48	0.81	0.97	0.48
H2GCN	0.86	0.83	0.63	0.84	0.71
MLP	0.72	0.51	0.48	0.80	0.60